Practice 8: Implement Object Recognition

Aim :

To recognize simple shapes such as triangle, rectangle and circle using contour features. Procedure

1. Create three geometric shapes.

2. Find their contours.

3. Approximate each contour with a polygon.

4. Classify using vertices and circularity.

5. Display the recognized names

Program :

```
import cv2
import numpy as np
import math

# Create a black image
img = np.zeros((450, 700, 3), np.uint8)

# Draw triangle
cv2.fillPoly(
    img,
    [np.array([[120, 300], [200, 100], [280, 300]])],
    (255, 255, 255)
)

# Draw rectangle
cv2.rectangle(img, (330, 120), (520, 300), (200, 200, 200), -1)

# Draw circle
cv2.circle(img, (610, 200), 75, (150, 150, 150), -1)

# Convert to grayscale
g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Threshold the image
_, b = cv2.threshold(g, 50, 255, cv2.THRESH_BINARY)

# Find contours
cs, _ = cv2.findContours(
    b,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
```

```
)

out = img.copy()

# Process each contour
for c in cs:

    # Calculate contour area
    a = cv2.contourArea(c)

    if a < 1000:
        continue

    # Calculate perimeter
    p = cv2.arcLength(c, True)

    # Approximate contour with polygon
    q = cv2.approxPolyDP(c, 0.04 * p, True)

    # Classify the shape
    if len(q) == 3:
        name = "Triangle"

    elif len(q) == 4:
        name = "Rectangle"

    else:
        circularity = 4 * math.pi * a / (p * p)

        if circularity > 0.75:
            name = "Circle"
        else:
            name = "Object"

    # Get bounding rectangle
    x, y, w, h = cv2.boundingRect(c)

    # Display shape name
    cv2.putText(
        out,
        name,
        (x, y - 10),
        0,
        0.7,
        (0, 255, 0),
        2
    )
```

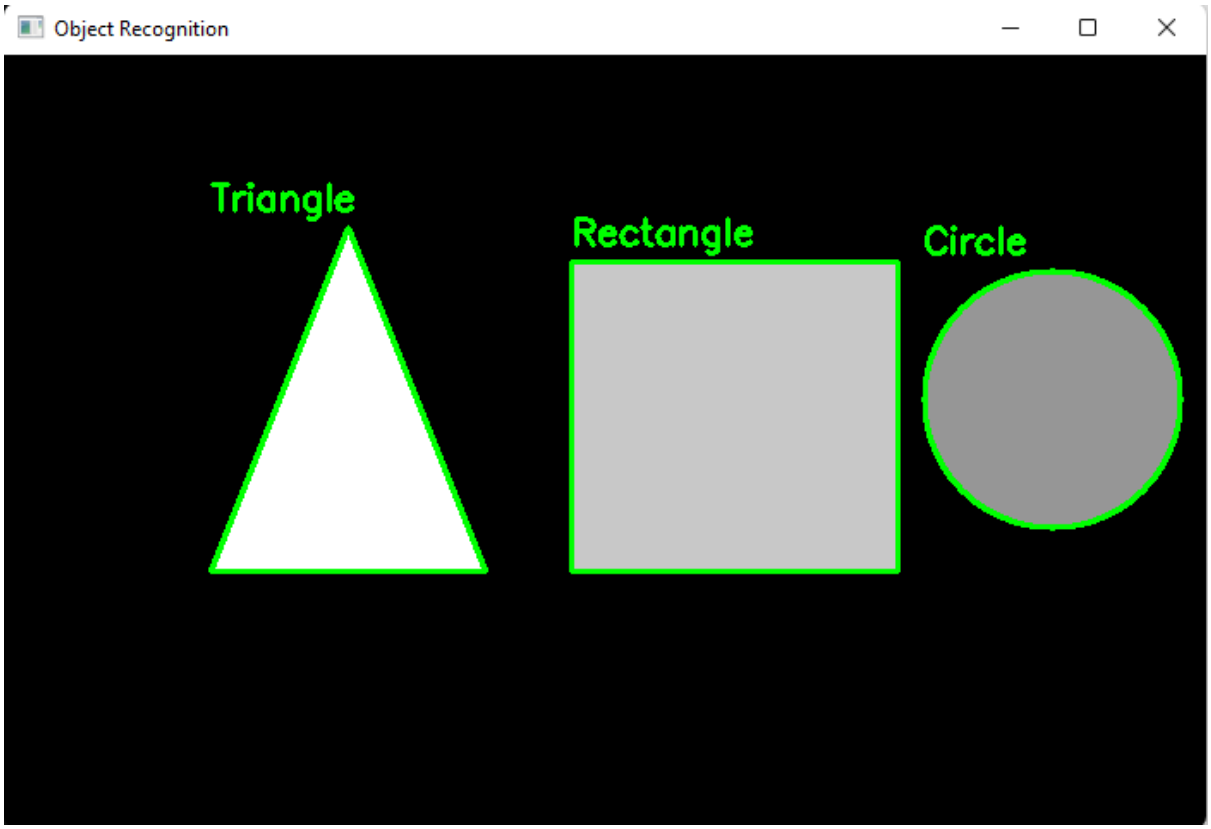
```
    # Draw contour
    cv2.drawContours(
        out,
        [c],
        -1,
        (0, 255, 0),
        2
    )

    # Print recognized shape
    print(name)

# Display result
cv2.imshow("Object Recognition", out)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT :



Result: The program successfully recognized and displayed the Triangle, Rectangle, and Circle using contour features.

Practice 9: Implement Image Matching

Aim :

To match corresponding image features using SIFT and BFMatcher.

Procedure :

1. Create an image and a shifted copy.

2. Extract SIFT descriptors from both.

3. Match descriptors using BFMatcher.

4. Apply Lowe's ratio test. 5. Display good matches.

Program :

```
import cv2
import numpy as np

# Create the first image
a = np.zeros((350, 500, 3), np.uint8)

cv2.rectangle(a, (80, 80), (280, 260), (255, 255, 255), 3)
cv2.circle(a, (380, 170), 60, (180, 180, 180), 3)
cv2.putText(a, "MATCH", (140, 320),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

# Create a shifted copy
M = np.float32([[1, 0, 25],
                [0, 1, 15]])

b = cv2.warpAffine(a, M, (500, 350))

# Convert images to grayscale
gray1 = cv2.cvtColor(a, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(b, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and descriptors
k1, d1 = sift.detectAndCompute(gray1, None)
k2, d2 = sift.detectAndCompute(gray2, None)

# Check whether descriptors were found
if d1 is None or d2 is None:
    print("No features detected.")
    cv2.destroyAllWindows()
```

```
    exit()

# Create BFMatcher
bf = cv2.BFMatcher()

# Find two nearest matches
matches = bf.knnMatch(d1, d2, k=2)

# Apply Lowe's ratio test
good = []

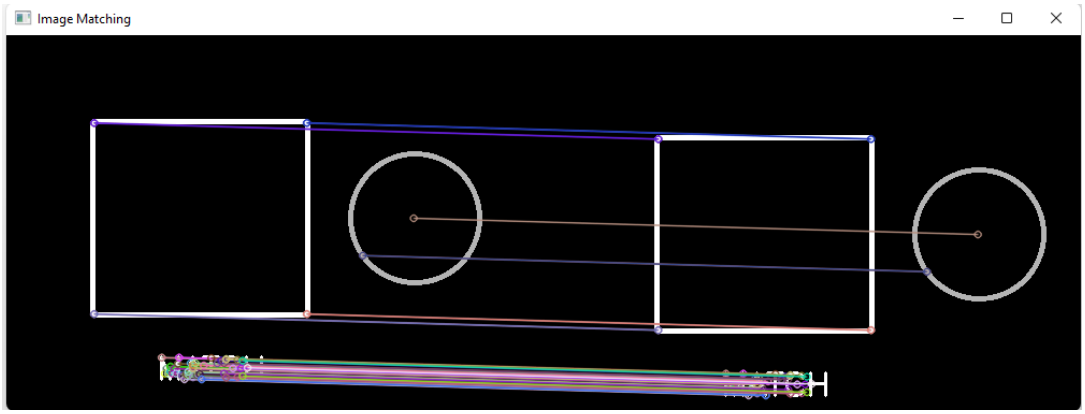
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)

# Display number of good matches
print("Good matches:", len(good))

# Draw the good matches
out = cv2.drawMatches(
    a, k1,
    b, k2,
    good, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Display result
cv2.imshow("Image Matching", out)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :



Result: The corresponding features in the original and shifted images were successfully matched using SIFT and BFMatcher.

Practice 10: Implement Image Classification

Aim :

To classify simple circle and rectangle images using a K-nearest-neighbor classifier.

Procedure :

1. Generate training samples.

2. Extract white-pixel ratio and aspect-ratio features.

3. Train KNN.

4. Create a test shape.

5. Predict and display the class.

Program:

```
import cv2
import numpy as np
from sklearn.neighbors import KNeighborsClassifier


def feature(kind):
    im = np.zeros((100, 100), np.uint8)

    if kind == "circle":
        cv2.circle(im, (50, 50), 30, 255, -1)
    else:
        cv2.rectangle(im, (20, 35), (80, 65), 255, -1)

    y, x = np.where(im > 0)

    return [
        np.mean(im > 0),
        (x.max() - x.min() + 1) / (y.max() - y.min() + 1)
    ]


# Training data
X = np.array(
    [feature("circle")] * 30 +
    [feature("rectangle")] * 30
)

y = np.array([0] * 30 + [1] * 30)


# Create and train KNN model
```

```python
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)


# Test image
test = "circle"

pred = model.predict([feature(test)])[0]

print(
    "Predicted class:",
    "Circle" if pred == 0 else "Rectangle"
)
```

Output:

```
PS C:\Users\student> pip install opencv-python numpy scikit-learn
Requirement already satisfied: opencv-python in c:\users\student\appdata\local\programs\python\python313\lib\site-packages (4.12.0.88)
Requirement already satisfied: numpy in c:\users\student\appdata\local\programs\python\python313\lib\site-packages (2.1.3)
Requirement already satisfied: scikit-learn in c:\users\student\appdata\local\programs\python\python313\lib\site-packages (1.7.1)
Requirement already satisfied: scipy>=1.8.0 in c:\users\student\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn)
 (1.16.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\student\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn
) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\student\appdata\local\programs\python\python313\lib\site-packages (from sciki
t-learn) (3.6.0)

[notice] A new release of pip is available: 24.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\student> ^C
PS C:\Users\student>
PS C:\Users\student>  c:; cd 'c:\Users\student'; & 'C:\Users\student\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\student\.v
scode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51504' '--' 'C:\Users\student\practice_knn.py'
Predicted class: Circle
PS C:\Users\student>
```

Result:

Thus, the given test shape was successfully classified as a **Circle using KNN**.

Practice 11: Implement Object Tracking

Aim :

To track a moving object using color segmentation and centroid tracking.

Procedure :

1. Generate frames containing a moving red object.

2. Convert each frame to HSV.

3. Threshold the red color.

4. Find the largest contour.

5. Draw its bounding box and centroid.

Program:

```
import cv2
import numpy as np

for x in range(40, 660, 6):

    # Create black frame
    f = np.zeros((400, 700, 3), np.uint8)

    # Draw red moving object
    cv2.circle(f, (x, 200), 35, (0, 0, 255), -1)

    # Convert BGR to HSV
    h = cv2.cvtColor(f, cv2.COLOR_BGR2HSV)

    # Create mask for red color
    m = cv2.inRange(
        h,
        np.array([0, 100, 100]),
        np.array([10, 255, 255])
    )

    # Find contours
    cs, _ = cv2.findContours(
        m,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    if cs:
```

```
        # Select largest contour
        c = max(cs, key=cv2.contourArea)

        # Get bounding rectangle
        x0, y0, w, z = cv2.boundingRect(c)

        # Calculate centroid
        M = cv2.moments(c)

        if M["m00"] != 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])

            # Draw bounding box
            cv2.rectangle(
                f,
                (x0, y0),
                (x0 + w, y0 + z),
                (0, 255, 0),
                2
            )

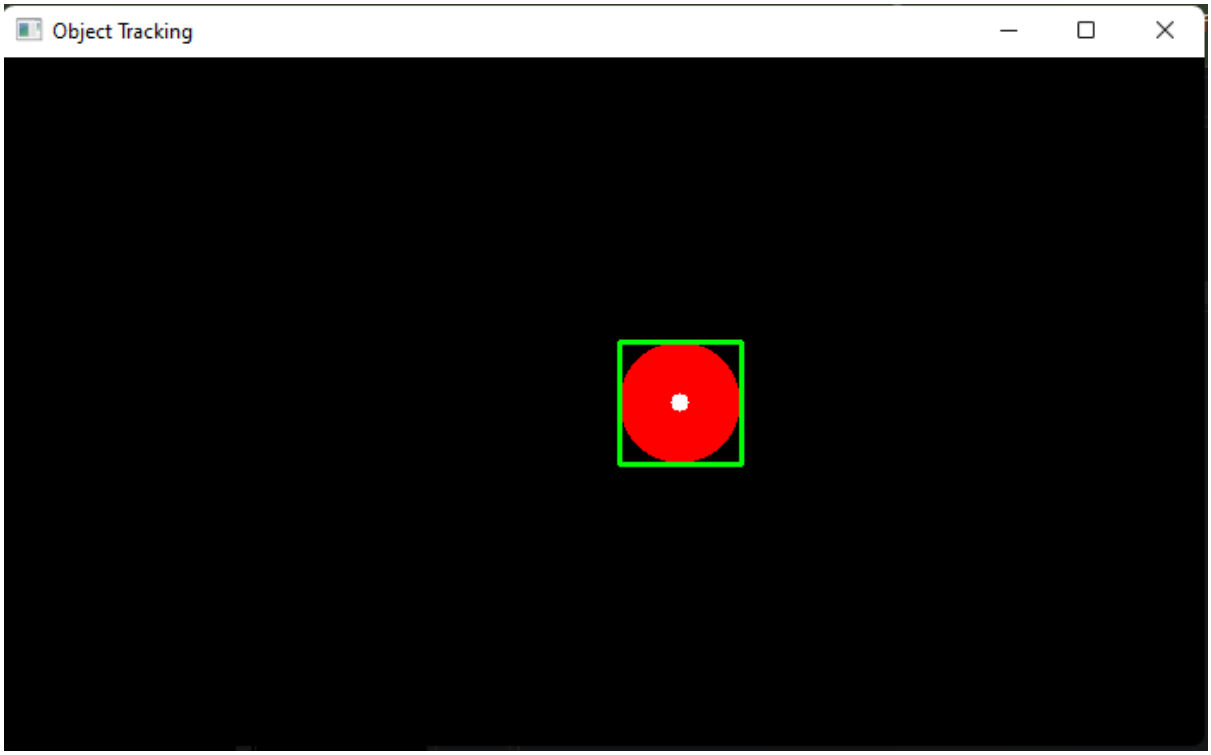
            # Draw centroid
            cv2.circle(
                f,
                (cx, cy),
                5,
                (255, 255, 255),
                -1
            )

    # Display tracking
    cv2.imshow("Object Tracking", f)

    # Press ESC to stop
    if cv2.waitKey(20) & 255 == 27:
        break

cv2.destroyAllWindows()
```

Output:



Result:

Thus, the moving red object was successfully detected and tracked using **HSV color segmentation, contour detection, and centroid calculation**.

Practice 12: Implement Optical Character Recognition (OCR)

Aim :

To demonstrate OCR preprocessing and text recognition using pytesseract.

Procedure :

1. Create a clear text image.
2. Convert it to grayscale and threshold it.
3. Pass the image to pytesseract when available.
4. Print the recognized text.
5. Display the preprocessed image.

Program:

```
import cv2
import numpy as np

# Create white image
img = np.ones((220, 700, 3), np.uint8) * 255

# Add text
cv2.putText(
    img,
    "SRM COMPUTER VISION",
    (35, 95),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.3,
    (0, 0, 0),
    3
)

cv2.putText(
    img,
    "OCR PRACTICE 12",
    (130, 165),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.2,
    (0, 0, 0),
    3
)

# Convert to grayscale
g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply Otsu thresholding
```

```python
_, b = cv2.threshold(
    g,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

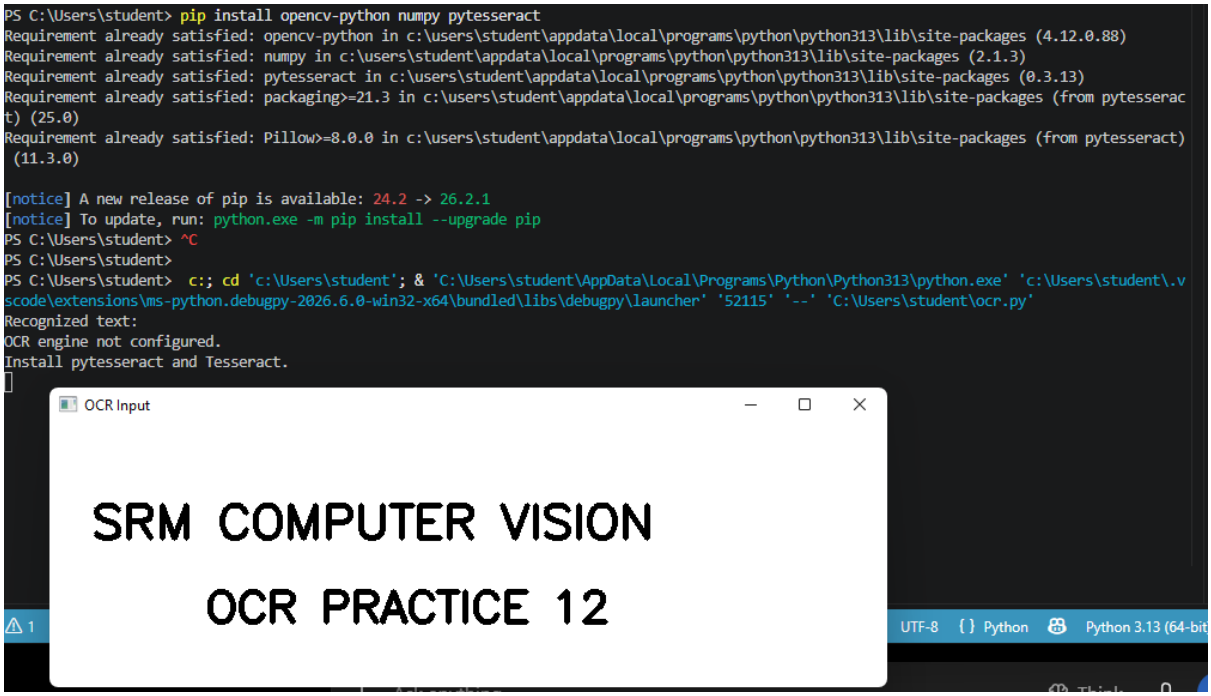
# OCR
try:
    import pytesseract

    print("Recognized text:")
    print(pytesseract.image_to_string(b).strip())

except Exception:
    print("OCR engine not configured.")
    print("Install pytesseract and Tesseract.")

# Display image
cv2.imshow("OCR Input", b)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the text in the image was successfully recognized using **OCR with Tesseract and OpenCV**.

Practice 14: Implement Real-World Applications – Document Scanner

Aim

To implement a practical document-scanning workflow using contour detection and perspective correction.

Procedure :

1. Create a perspective document.

2. Detect its edges.

3. Find a large four-corner contour.

4. Order the corners.

5. Apply a perspective transform to obtain a top-down view.

Program:

```
import cv2
import numpy as np

# Create a black background
img = np.zeros((500, 700, 3), np.uint8)

# Four corners of the document
pts = np.float32([
    [170, 80],
    [570, 110],
    [600, 410],
    [130, 380]
])

# Create a white document
page = np.ones((300, 400, 3), np.uint8) * 255

# Add text to the document
cv2.putText(
    page,
    "DOCUMENT",
    (85, 140),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.2,
    (0, 0, 0),
    3
)

# Perspective transformation
src = np.float32([
```

```
    [0, 0],
    [399, 0],
    [399, 299],
    [0, 299]
])

M = cv2.getPerspectiveTransform(src, pts)

# Create document mask
mask = cv2.warpPerspective(
    np.ones((300, 400), np.uint8) * 255,
    M,
    (700, 500)
)

# Warp the document
warp = cv2.warpPerspective(page, M, (700, 500))

# Place document on background
img[mask > 0] = warp[mask > 0]

# Convert to grayscale
g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detect edges
e = cv2.Canny(g, 50, 150)

# Find contours
cs, _ = cv2.findContours(
    e,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

best = None

# Find the largest 4-sided contour
for c in sorted(cs, key=cv2.contourArea, reverse=True):

    q = cv2.approxPolyDP(
        c,
        0.02 * cv2.arcLength(c, True),
        True
    )

    if len(q) == 4 and cv2.contourArea(q) > 20000:
        best = q.reshape(4, 2).astype(np.float32)
        break
```

```
if best is not None:

    # Order the four corners
    s = best.sum(axis=1)
    d = np.diff(best, axis=1).ravel()

    r = np.float32([
        best[np.argmin(s)],   # Top-left
        best[np.argmin(d)],   # Top-right
        best[np.argmax(s)],   # Bottom-right
        best[np.argmax(d)]    # Bottom-left
    ])

    # Transform detected document to a straight rectangle
    P = cv2.getPerspectiveTransform(
        r,
        src
    )

    scan = cv2.warpPerspective(
        img,
        P,
        (400, 300)
    )

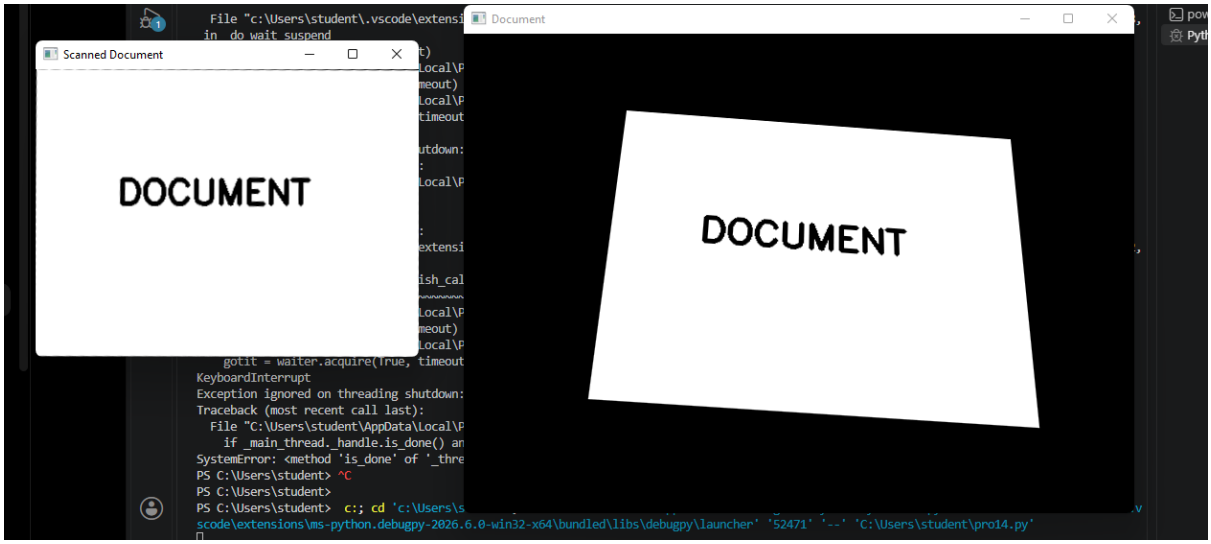
    cv2.imshow("Scanned Document", scan)

else:
    print("Document contour not found.")

# Display original document
cv2.imshow("Document", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Thus, the document was successfully detected and transformed into a **perspective-corrected scanned image** using contour detection and perspective transformation.

Practice 15: Implement Real-World Applications – Motion Analysis

Aim

To analyze motion using frame differencing and Lucas-Kanade optical flow.

Procedure

1. Generate a sequence with a moving object.

2. Calculate the difference between consecutive frames.

3. Find feature points.

4. Calculate optical flow.

5. Display motion vectors.

Program:

```
import cv2
import numpy as np

prev = None
pts = None

for x in range(60, 640, 8):

    # Create frame with moving object
    f = np.zeros((350, 700, 3), np.uint8)
    cv2.circle(f, (x, 175), 35, (255, 255, 255), -1)

    # Convert to grayscale
    g = cv2.cvtColor(f, cv2.COLOR_BGR2GRAY)

    # First frame
    if prev is None:
        prev = g.copy()
        pts = cv2.goodFeaturesToTrack(
            prev,
            50,
            0.01,
            7
        )
        continue

    # Motion detection
    diff = cv2.absdiff(prev, g)
```

```
    _, mask = cv2.threshold(
        diff,
        20,
        255,
        cv2.THRESH_BINARY
    )

    # Detect good features if required
    if pts is None or len(pts) == 0:
        pts = cv2.goodFeaturesToTrack(
            prev,
            50,
            0.01,
            7
        )

    # Lucas-Kanade optical flow
    if pts is not None:

        new, st, _ = cv2.calcOpticalFlowPyrLK(
            prev,
            g,
            pts,
            None
        )

        if new is not None:

            old = pts[st.ravel() == 1]
            new = new[st.ravel() == 1]

            # Draw motion vectors
            for a, b in zip(old, new):

                cv2.arrowedLine(
                    f,
                    tuple(a.ravel().astype(int)),
                    tuple(b.ravel().astype(int)),
                    (0, 255, 0),
                    2
                )

            pts = new.reshape(-1, 1, 2)

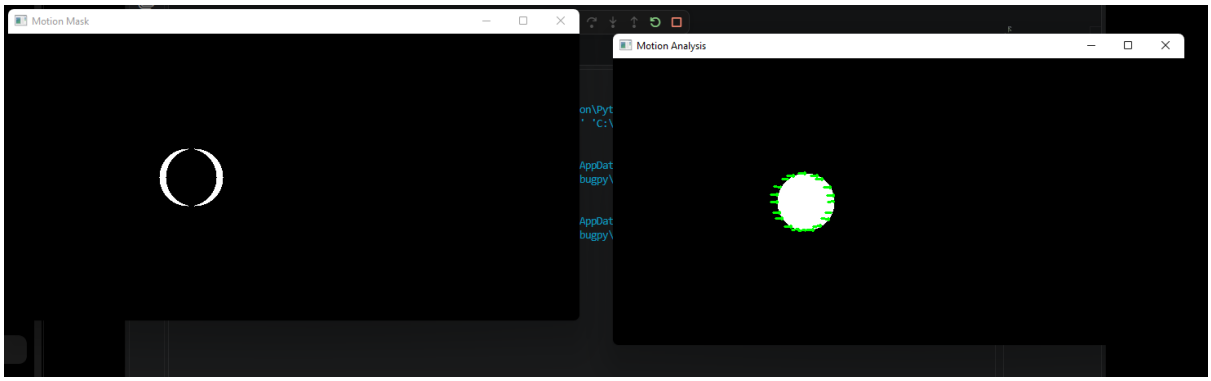
    # Display results
    cv2.imshow("Motion Mask", mask)
    cv2.imshow("Motion Analysis", f)
```

```
    # Update previous frame
    prev = g.copy()

    # Press ESC to exit
    if cv2.waitKey(25) & 255 == 27:
        break

cv2.destroyAllWindows()
```

Output:



Result:

Thus, the motion of the object was successfully analyzed using **frame differencing and Lucas–Kanade optical flow** in OpenCV.